

Data Exfil through DNS Tunnelling

Key Points

DNS uses UDP (mostly).

DNS is normally allowed through firewalls and gateways.

- Why attacker use DNS for Exfiltration

DNS is an always on service.

Flexible payload – data can be encoded into subdomain levels.

High cover – Queries hid in plain sight unless inspected closely.

Indicators of attack

Many DNS queries are sent to a single external domain.

Long subdomain labels.

High entropy or Base32/Base64.

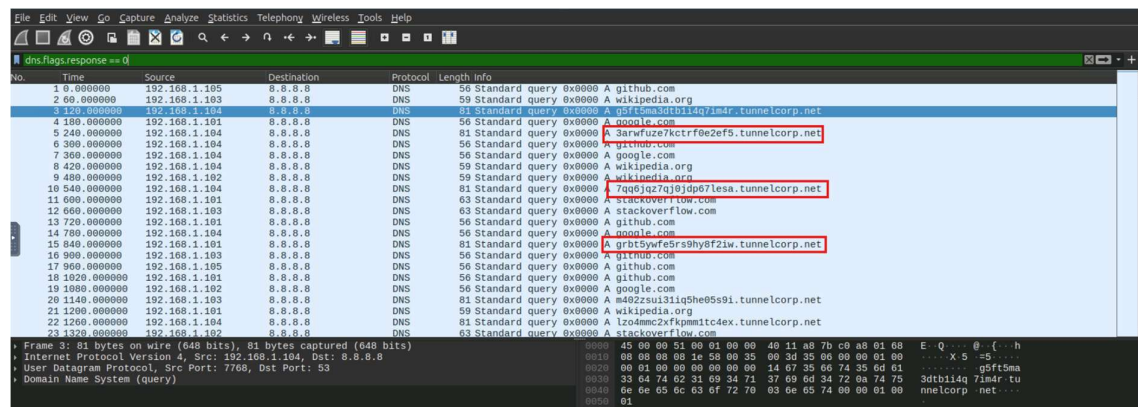
Rare record types (TXT, NULL).

Unusual response behaviour

Queries at regular intervals (beaconing behaviour).

Method 1 (PCAP)

1. - I filtered through the traffic using the “dns” command to see just DNS queries.
- Then used the “dns.flags.response == 0” to filter DNS queries with no response.
- The out put shows some queries with a large query length



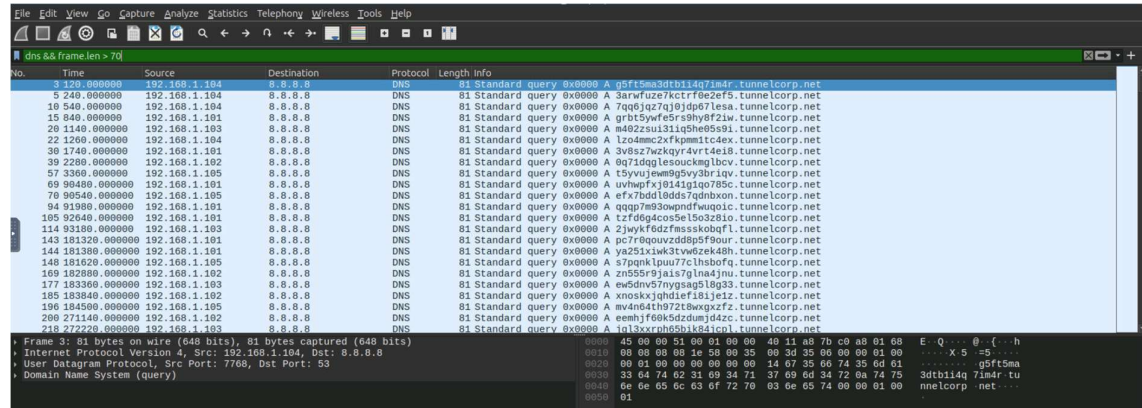
The image shows a Wireshark packet capture window with a filter applied: `dns.flags.response == 0`. The packet list shows several DNS standard queries. The packet details pane for the selected packet (No. 18) shows the query structure. The packet bytes pane shows the raw data of the query, which is a long subdomain followed by a standard query format.

No.	Time	Source	Destination	Protocol	Length	Info
16	0.000000	192.168.1.105	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
20	0.000000	192.168.1.103	8.8.8.8	DNS	59	Standard query 0x0000 A wikipedia.org
18	0.000000	192.168.1.105	8.8.8.8	DNS	56	Standard query 0x0000 A y0d4cm3i0p3d9j1e4r.tunne1corp.net
4	100.000000	192.168.1.101	8.8.8.8	DNS	56	Standard query 0x0000 A google.com
5	240.000000	192.168.1.104	8.8.8.8	DNS	81	Standard query 0x0000 A 3arufuze7kctrf0e2ef5.tunne1corp.net
6	300.000000	192.168.1.104	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
7	360.000000	192.168.1.104	8.8.8.8	DNS	56	Standard query 0x0000 A google.com
8	420.000000	192.168.1.104	8.8.8.8	DNS	59	Standard query 0x0000 A wikipedia.org
9	480.000000	192.168.1.102	8.8.8.8	DNS	59	Standard query 0x0000 A wikipedia.org
10	540.000000	192.168.1.104	8.8.8.8	DNS	81	Standard query 0x0000 A 7qq6jqz7qj9dp67lesa.tunne1corp.net
11	600.000000	192.168.1.101	8.8.8.8	DNS	63	Standard query 0x0000 A stackoverflow.com
12	660.000000	192.168.1.103	8.8.8.8	DNS	63	Standard query 0x0000 A stackoverflow.com
13	720.000000	192.168.1.101	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
14	780.000000	192.168.1.104	8.8.8.8	DNS	56	Standard query 0x0000 A google.com
15	840.000000	192.168.1.101	8.8.8.8	DNS	81	Standard query 0x0000 A grbt5ywe5rs9hy8f2iw.tunne1corp.net
16	900.000000	192.168.1.103	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
17	960.000000	192.168.1.105	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
18	1020.000000	192.168.1.101	8.8.8.8	DNS	56	Standard query 0x0000 A github.com
19	1080.000000	192.168.1.102	8.8.8.8	DNS	56	Standard query 0x0000 A google.com
20	1140.000000	192.168.1.103	8.8.8.8	DNS	81	Standard query 0x0000 A m402zsu131q5he05s91.tunne1corp.net
21	1200.000000	192.168.1.101	8.8.8.8	DNS	59	Standard query 0x0000 A wikipedia.org
22	1260.000000	192.168.1.104	8.8.8.8	DNS	81	Standard query 0x0000 A l2o4mc2xfpmitc4ex.tunne1corp.net
23	1320.000000	192.168.1.102	8.8.8.8	DNS	63	Standard query 0x0000 A stackoverflow.com

Frame 3: 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface 0
Internet Protocol Version 4, Src: 192.168.1.104, Dst: 8.8.8.8
User Datagram Protocol, Src Port: 7760, Dst Port: 53
Domain Name System (query)
0000 45 00 00 51 00 01 00 00 40 11 a8 7b c0 a8 01 08 E . Q . . . @ . (. . . h
0010 08 00 00 08 1e 50 00 35 00 3d 35 00 00 01 00 X 5 . - S
0020 00 01 00 00 00 00 00 00 14 67 35 66 74 35 6d 01 g5ft5ma
0030 33 64 74 62 31 69 34 71 37 69 6d 34 72 0a 74 75 3dtb1i4g 7im4r tu
0040 6e 6e 65 6c 63 6f 72 70 03 6e 65 74 00 00 01 00 nnelcorp .net
0050 01

2. - I used the filter dns && frame.len > 70 to narrow down query lengths greater than 70.

- With this we can see one suspicious domain which is receiving multiple dns requests



The screenshot shows a Wireshark packet capture with the filter 'dns && frame.len > 70'. The packet list displays 70 DNS queries, all of which are standard queries to the domain 'tunnelcorp.net'. The packet details pane for the selected packet (No. 31) shows the following structure:

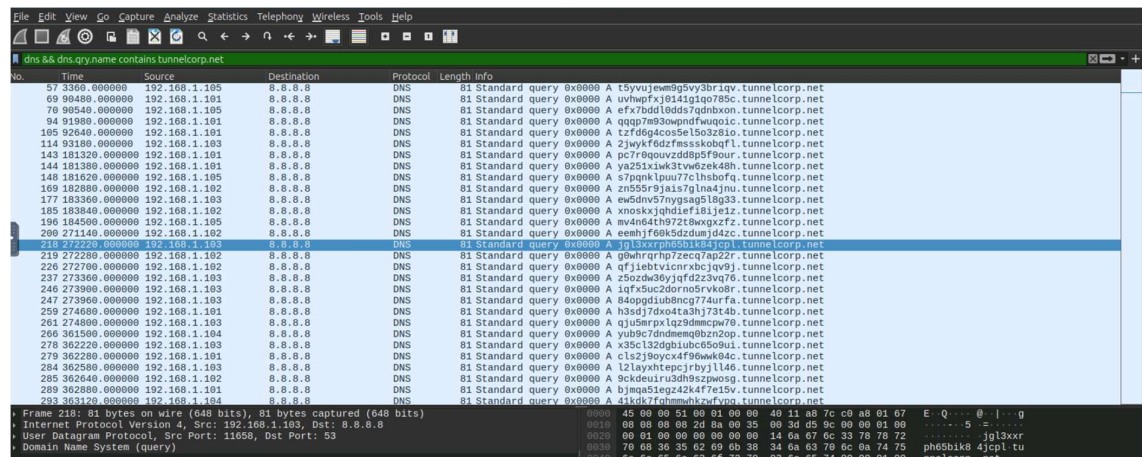
- Frame 31: 81 bytes on wire (648 bits), 81 bytes captured (648 bits)
- Internet Protocol Version 4, Src: 192.168.1.104, Dst: 8.8.8.8
- User Datagram Protocol, Src Port: 7768, Dst Port: 53
- Domain Name System (query)

The packet bytes pane shows the raw data of the DNS query, including the transaction ID, flags, and the query name 'tunnelcorp.net'.

3. - I then filtered the domain using “dns && dns.qry.name contains tunnelcorp.net”

- with this we have identified the dns tunnelling attempt.

- compromised hosts are sending data in chunks using dns tunnelling techniques and there is only 1 external domain receiving these queries (tunnelcorp.net)



The screenshot shows a Wireshark packet capture with the filter 'dns && dns.qry.name contains tunnelcorp.net'. The packet list displays 70 DNS queries, all of which are standard queries to the domain 'tunnelcorp.net'. The packet details pane for the selected packet (No. 31) shows the following structure:

- Frame 31: 81 bytes on wire (648 bits), 81 bytes captured (648 bits)
- Internet Protocol Version 4, Src: 192.168.1.104, Dst: 8.8.8.8
- User Datagram Protocol, Src Port: 7768, Dst Port: 53
- Domain Name System (query)

The packet bytes pane shows the raw data of the DNS query, including the transaction ID, flags, and the query name 'tunnelcorp.net'.

4. – We can do the same analysis as well using splunk.
 - sourcetype=DNS_logs to view the data but we want to see how many request each compromised IP sent for this we use the
 - index="data_exfil" sourcetype="DNS_logs" | stats count by src_ip

Search | Splunk 10.0.0

index=data_exfil sourcetype=dns_logs | stats count by src_ip

2,000 events (before 5/15/26 6:15:33.000 PM) No Event Sampling

src_ip	count
192.168.1.101	486
192.168.1.102	378
192.168.1.103	485
192.168.1.104	413
192.168.1.105	486

5. Then to filter further
 - index="data_exfil" sourcetype="dns_logs" | stats count by query | sort -count

Search | Splunk 10.0.0

index="data_exfil" sourcetype="dns_logs" | stats count by query | sort -count

2,000 events (before 5/15/26 6:23:41.000 PM) No Event Sampling

query	count
microsoft.com	361
stackoverflow.com	339
wikipedia.org	337
github.com	331
google.com	317
@36er95gzqabe3ymiht.tunnelcorp.net	1
@apj5ix7azdeikfa5wjt.tunnelcorp.net	1
@fvgpdsiybcma94rq7e.tunnelcorp.net	1
@e3uxie9b84dzfbg3b12.tunnelcorp.net	1

6. But we still can go a step further we can use index="data_exfil" sourcetype="DNS_logs" | where len(query) > 30

Search | Splunk 10.0.0

index="data_exfil" sourcetype="DNS_logs" | where len(query) > 30

2,000 events (before 5/15/26 6:23:41.000 PM) No Event Sampling

query	count
@36er95gzqabe3ymiht.tunnelcorp.net	1
@apj5ix7azdeikfa5wjt.tunnelcorp.net	1
@fvgpdsiybcma94rq7e.tunnelcorp.net	1
@e3uxie9b84dzfbg3b12.tunnelcorp.net	1
@ieza5rfdm96q2rpja10.tunnelcorp.net	1
@jrc5sr5zd155984gobm.tunnelcorp.net	1
@ophqaiwfb05jxnrdby.tunnelcorp.net	1
@q7ldaglesouckmg1bcv.tunnelcorp.net	1
@tdq1l5lqxn8mdk6769.tunnelcorp.net	1
@ymnk34x9z849zhosm.tunnelcorp.net	1

Confirmation DNS tunnelling with the above indicators via Splunk approach

7. We can analyse this data as well via the .log file in the terminal

- head dns_exfil.log

```
File Edit View Search Terminal Help
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$ ls
dns_exfil.log  dns_exfil.pcap
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$ head dns_exfil.log
timestamp=2025-09-01T00:00:00 src_ip=192.168.1.105 dst_ip=8.8.8.8 protocol=DNS query=github.com label=benign
timestamp=2025-09-01T00:01:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=wikipedia.org label=benign
timestamp=2025-09-01T00:02:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=g5ft5ma3dtb1i4q7im4r.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:03:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=google.com label=benign
timestamp=2025-09-01T00:04:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=3arwfuze7kctrf0e2ef5.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:05:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=github.com label=benign
timestamp=2025-09-01T00:06:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=google.com label=benign
timestamp=2025-09-01T00:07:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=wikipedia.org label=benign
timestamp=2025-09-01T00:08:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=wikipedia.org label=benign
timestamp=2025-09-01T00:09:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=7qq6jqz7qj0jdp67lesa.tunnelcorp.net label=suspicious
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$
```

8. I now used the cat and grep command to filter down further

- cat dns_exfil.log | grep suspicious

```
File Edit View Search Terminal Help
timestamp=2025-09-01T00:04:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=3arwfuze7kctrf0e2ef5.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:05:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=github.com label=benign
timestamp=2025-09-01T00:06:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=google.com label=benign
timestamp=2025-09-01T00:07:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=wikipedia.org label=benign
timestamp=2025-09-01T00:08:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=wikipedia.org label=benign
timestamp=2025-09-01T00:09:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=7qq6jqz7qj0jdp67lesa.tunnelcorp.net label=suspicious
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$ cat dns_exfil.log | grep suspicious
timestamp=2025-09-01T00:02:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=g5ft5ma3dtb1i4q7im4r.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:04:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=3arwfuze7kctrf0e2ef5.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:09:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=7qq6jqz7qj0jdp67lesa.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:14:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=grbt5ywf5rs9hy8f2lw.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:19:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=m402zsu311q5he05s9i.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:21:00 src_ip=192.168.1.104 dst_ip=8.8.8.8 protocol=DNS query=lzo4mmc2xfkpm1tc4ex.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:29:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=3v8sz7wzkqyr4vrt4e18.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:38:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=0q71dgglesoucknglbcv.tunnelcorp.net label=suspicious
timestamp=2025-09-01T00:56:00 src_ip=192.168.1.105 dst_ip=8.8.8.8 protocol=DNS query=t5yvu9jewm9g5vy3br1qv.tunnelcorp.net label=suspicious
timestamp=2025-09-02T01:08:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=uvhwpxfj0141g1q0785c.tunnelcorp.net label=suspicious
timestamp=2025-09-02T01:09:00 src_ip=192.168.1.105 dst_ip=8.8.8.8 protocol=DNS query=efx7bddl0dds7qdnbxon.tunnelcorp.net label=suspicious
timestamp=2025-09-02T01:33:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=qqq7m930wpndfwuqoic.tunnelcorp.net label=suspicious
timestamp=2025-09-02T01:44:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=tzfd6g4cos5el5o3z8io.tunnelcorp.net label=suspicious
timestamp=2025-09-02T01:53:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=2jwykf6dzfnssskobqfl.tunnelcorp.net label=suspicious
timestamp=2025-09-03T02:22:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=pc7r0qouvd8p5f9our.tunnelcorp.net label=suspicious
timestamp=2025-09-03T02:23:00 src_ip=192.168.1.101 dst_ip=8.8.8.8 protocol=DNS query=ya251xlwk3tw62ek48h.tunnelcorp.net label=suspicious
timestamp=2025-09-03T02:27:00 src_ip=192.168.1.105 dst_ip=8.8.8.8 protocol=DNS query=s7pgnklpuu77clhsb0fq.tunnelcorp.net label=suspicious
timestamp=2025-09-03T02:48:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=zn555r9jai57glna4jnu.tunnelcorp.net label=suspicious
timestamp=2025-09-03T02:56:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=ew5dnv57nygsag5l8g33.tunnelcorp.net label=suspicious
timestamp=2025-09-03T03:04:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=xnoskxjghdief18ije1z.tunnelcorp.net label=suspicious
timestamp=2025-09-03T03:15:00 src_ip=192.168.1.105 dst_ip=8.8.8.8 protocol=DNS query=mv4n64th972t8wxgzfz.tunnelcorp.net label=suspicious
timestamp=2025-09-04T03:19:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=eenhj6k5dzdumdj4zc.tunnelcorp.net label=suspicious
timestamp=2025-09-04T03:37:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=jgl3xxrph65bik84jcp1.tunnelcorp.net label=suspicious
timestamp=2025-09-04T03:38:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=g0whrqrhp7zecz7ap22r.tunnelcorp.net label=suspicious
timestamp=2025-09-04T03:45:00 src_ip=192.168.1.102 dst_ip=8.8.8.8 protocol=DNS query=qfjietv1cnrxcjqv9j.tunnelcorp.net label=suspicious
timestamp=2025-09-04T03:56:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=z5ozdw36yjqfd23vq76.tunnelcorp.net label=suspicious
timestamp=2025-09-04T04:05:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=lqfx5uc2dorno5vko8r.tunnelcorp.net label=suspicious
timestamp=2025-09-04T04:06:00 src_ip=192.168.1.103 dst_ip=8.8.8.8 protocol=DNS query=w4opndub8oc774urfa.tunnelcorp.net label=suspicious
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$
```

9. Using the cat dns_exfil.log | grep suspicious | cut -d ' ' -f2 | sort | uniq -c | sort -nr to count suspicious activity per host.

```
src_ip=192.168.1.104
src_ip=192.168.1.105
src_ip=192.168.1.103
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$ cat dns_exfil.log | grep suspicious | cut -d ' ' -f2 | sort | uniq -c | sort -nr
 72 src_ip=192.168.1.103
 63 src_ip=192.168.1.105
 62 src_ip=192.168.1.104
 60 src_ip=192.168.1.102
 58 src_ip=192.168.1.101
ubuntu@tryhackme:~/Desktop/data_exfil/dns_exfil$
```

10. Above we can see 192.168.1.103 sent the most requests to the suspicious domain

Data Exfiltration through FTP

Key Points

Adversaries use legitimate FTP servers, public or misconfigured internal servers.

Compromised Credentials.

Non-standard ports or tunnelling to blend with traffic.

Indicators of attack

USER or PASS commands cleartext credentials

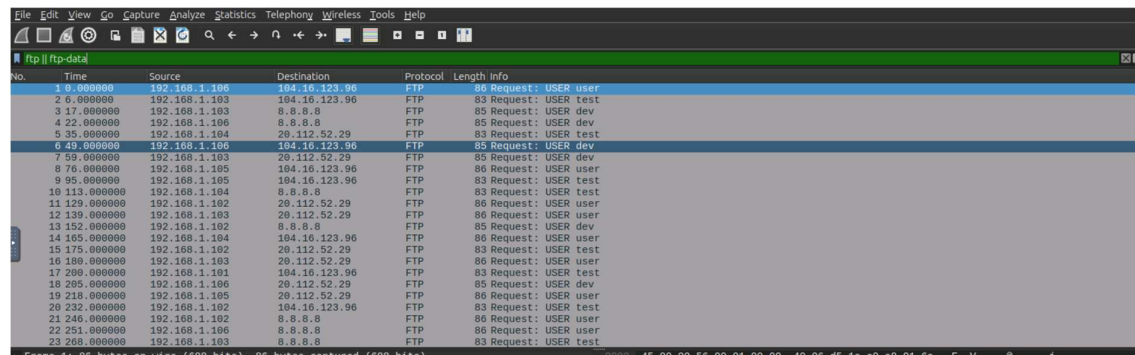
STOR (upload) and RETR (download) commands repeated

Large data connections to external IPs

Data channel openings on ephemeral ports paired with large payloads.

Analysis through Wireshark

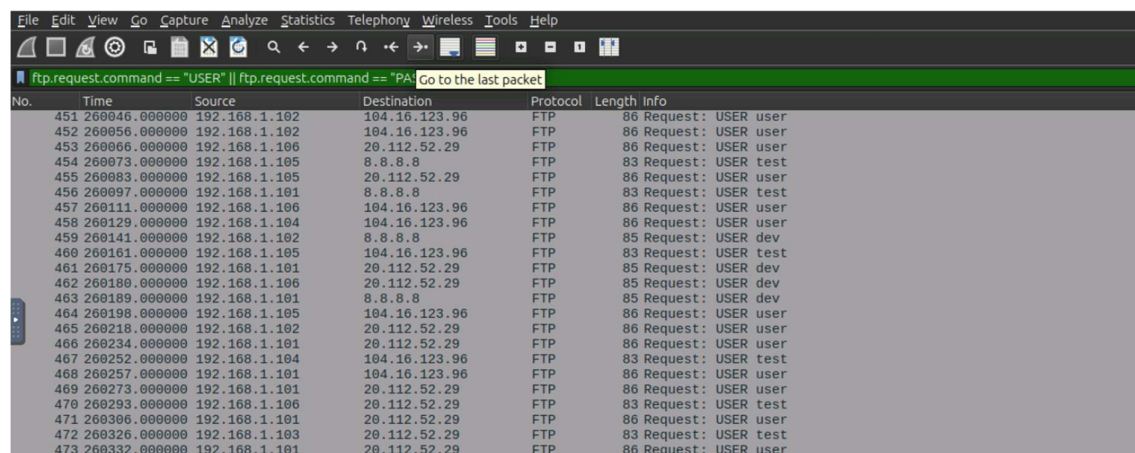
1. We filter the traffic the get FTP control traffic only ftp || ftp-data



The screenshot shows the Wireshark interface with the filter 'ftp || ftp-data' applied. The packet list displays 23 packets, all of which are FTP control traffic. The packet details pane shows the structure of an FTP request, including the command, user, and password fields.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.106	104.16.123.96	FTP	83	Request: USER user
2	0.000000	192.168.1.103	104.16.123.96	FTP	83	Request: USER user
3	0.000000	192.168.1.103	8.8.8.8	FTP	85	Request: USER dev
4	0.000000	192.168.1.106	8.8.8.8	FTP	85	Request: USER dev
5	0.000000	192.168.1.104	20.112.52.29	FTP	83	Request: USER test
6	0.000000	192.168.1.106	104.16.123.96	FTP	85	Request: USER dev
7	0.000000	192.168.1.103	20.112.52.29	FTP	85	Request: USER dev
8	0.000000	192.168.1.105	104.16.123.96	FTP	86	Request: USER user
9	0.000000	192.168.1.105	104.16.123.96	FTP	83	Request: USER test
10	0.000000	192.168.1.104	8.8.8.8	FTP	83	Request: USER test
11	0.000000	192.168.1.102	20.112.52.29	FTP	86	Request: USER user
12	0.000000	192.168.1.103	20.112.52.29	FTP	86	Request: USER user
13	0.000000	192.168.1.102	8.8.8.8	FTP	85	Request: USER dev
14	0.000000	192.168.1.104	104.16.123.96	FTP	86	Request: USER user
15	0.000000	192.168.1.102	20.112.52.29	FTP	83	Request: USER test
16	0.000000	192.168.1.103	20.112.52.29	FTP	86	Request: USER user
17	0.000000	192.168.1.101	104.16.123.96	FTP	83	Request: USER test
18	0.000000	192.168.1.106	20.112.52.29	FTP	85	Request: USER dev
19	0.000000	192.168.1.105	20.112.52.29	FTP	86	Request: USER user
20	0.000000	192.168.1.102	104.16.123.96	FTP	83	Request: USER test
21	0.000000	192.168.1.102	8.8.8.8	FTP	86	Request: USER user
22	0.000000	192.168.1.106	8.8.8.8	FTP	86	Request: USER user
23	0.000000	192.168.1.103	8.8.8.8	FTP	83	Request: USER test

2. Now we can filter to show login attempts with USER/PASS using ftp.request.command == "USER" || ftp.request.command == "PASS"

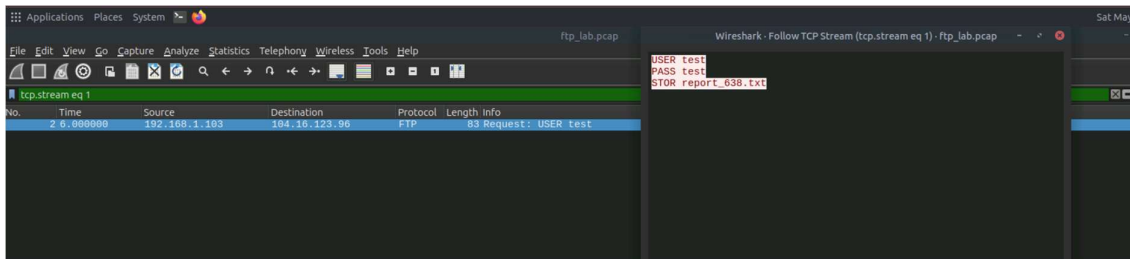


The screenshot shows the Wireshark interface with the filter 'ftp.request.command == "USER" || ftp.request.command == "PASS"' applied. The packet list displays 23 packets, all of which are login attempts. The packet details pane shows the structure of an FTP request, including the command, user, and password fields.

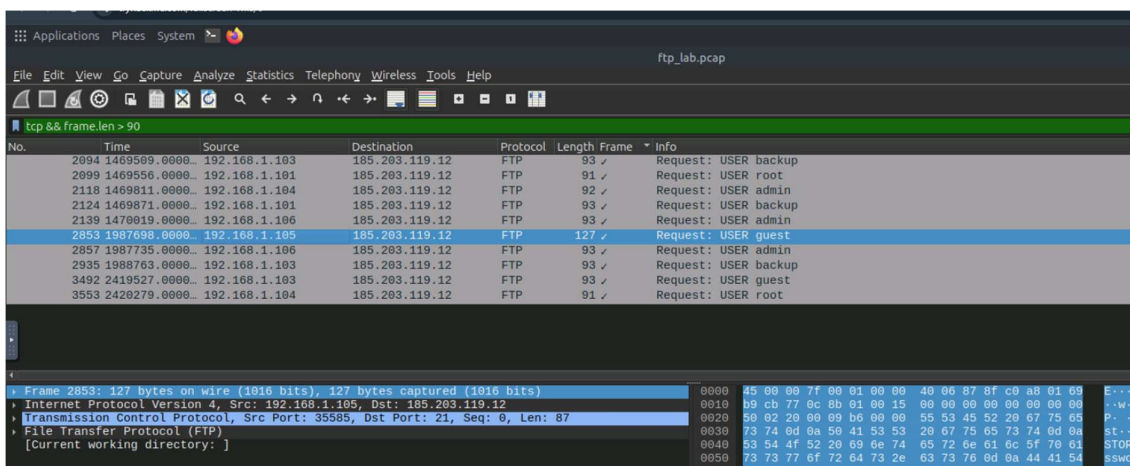
No.	Time	Source	Destination	Protocol	Length	Info
451	260046.000000	192.168.1.102	104.16.123.96	FTP	86	Request: USER user
452	260056.000000	192.168.1.102	104.16.123.96	FTP	86	Request: USER user
453	260066.000000	192.168.1.106	20.112.52.29	FTP	86	Request: USER user
454	260073.000000	192.168.1.105	8.8.8.8	FTP	83	Request: USER test
455	260083.000000	192.168.1.105	20.112.52.29	FTP	86	Request: USER user
456	260097.000000	192.168.1.101	8.8.8.8	FTP	83	Request: USER test
457	260111.000000	192.168.1.106	104.16.123.96	FTP	86	Request: USER user
458	260129.000000	192.168.1.104	104.16.123.96	FTP	86	Request: USER user
459	260141.000000	192.168.1.102	8.8.8.8	FTP	85	Request: USER dev
460	260161.000000	192.168.1.105	104.16.123.96	FTP	83	Request: USER test
461	260175.000000	192.168.1.101	20.112.52.29	FTP	85	Request: USER dev
462	260180.000000	192.168.1.106	20.112.52.29	FTP	85	Request: USER dev
463	260189.000000	192.168.1.101	8.8.8.8	FTP	85	Request: USER dev
464	260198.000000	192.168.1.105	104.16.123.96	FTP	86	Request: USER user
465	260218.000000	192.168.1.102	20.112.52.29	FTP	86	Request: USER user
466	260234.000000	192.168.1.101	20.112.52.29	FTP	86	Request: USER user
467	260252.000000	192.168.1.104	104.16.123.96	FTP	83	Request: USER test
468	260257.000000	192.168.1.101	104.16.123.96	FTP	86	Request: USER user
469	260273.000000	192.168.1.101	20.112.52.29	FTP	86	Request: USER user
470	260293.000000	192.168.1.106	20.112.52.29	FTP	83	Request: USER test
471	260306.000000	192.168.1.101	20.112.52.29	FTP	86	Request: USER user
472	260326.000000	192.168.1.103	20.112.52.29	FTP	83	Request: USER test
473	260332.000000	192.168.1.101	20.112.52.29	FTP	86	Request: USER user

3. We can now use ftp contains “STOR” to look for anomalies in file names or credentials.

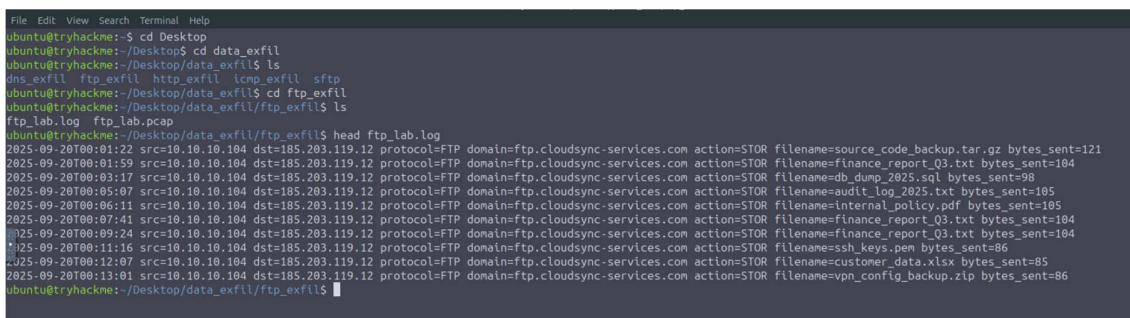
I then followed the tcp stream to look for suspicious files like PDF, csv, TXT etc.



4. We can then use the ftp && frame.len > 90 to see traffic with large payloads



Analysis Via Terminal



2. in the above image we can see the STOR action now we will filter so we can see uploads per source host using 'cat ftp_lab.log | cut -d' ' -f2 | sort | uniq -c | sort -nr'



3. `cat ftp_lab.log | cut -d' ' -f7` was used to grab the files uploaded.

```
ubuntu@tryhackme:~/Desktop/data_exfil/ftp_exfil$ cat ftp_lab.log | cut -d' ' -f2 | sort | uniq -c | sort -nr
400 src=10.10.10.104
ubuntu@tryhackme:~/Desktop/data_exfil/ftp_exfil$ cat ftp_lab.log | cut -d' ' -f7
filename=source_code_backup.tar.gz
filename=finance_report_Q3.txt
filename=db_dump_2025.sql
filename=audit_log_2025.txt
filename=internal_policy.pdf
filename=finance_report_Q3.txt
filename=finance_report_Q3.txt
filename=ssh_keys.pem
filename=customer_data.xlsx
filename=vpn_config_backup.zip
filename=customer_data.xlsx
filename=vpn_config_backup.zip
filename=THM{ftp_hidden_fl4g}.txt
filename=customer_data.xlsx
filename=internal_policy.pdf
filename=incident_notes.docx
filename=finance_report_Q3.txt
filename=internal_policy.pdf
filename=internal_policy.pdf
```

4. Used `grep` to filter for sensitive uploads `cat ftp_lab.log | grep -E "ssh|db|backup|customer"`

```
File Edit View Search Terminal Help
ubuntu@tryhackme:~/Desktop/data_exfil/ftp_exfil$ cat ftp_lab.log | grep -E "ssh|db|backup|customer"
2025-09-20T00:01:22 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=source_code_backup.tar.gz bytes_sent=121
2025-09-20T00:03:17 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=db_dump_2025.sql bytes_sent=98
2025-09-20T00:11:16 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=ssh_keys.pem bytes_sent=86
2025-09-20T00:12:07 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=customer_data.xlsx bytes_sent=85
2025-09-20T00:13:01 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-20T00:13:48 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=customer_data.xlsx bytes_sent=85
2025-09-20T00:15:26 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-20T00:18:37 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=customer_data.xlsx bytes_sent=85
2025-09-20T00:24:04 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-21T00:06:50 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-21T00:13:02 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=ssh_keys.pem bytes_sent=86
2025-09-21T00:15:22 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-21T00:17:48 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=customer_data.xlsx bytes_sent=85
2025-09-22T00:01:11 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=ssh_keys.pem bytes_sent=86
2025-09-22T00:03:35 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=db_dump_2025.sql bytes_sent=98
2025-09-22T00:07:48 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-22T00:08:32 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
2025-09-22T00:12:34 src=10.10.10.104 dst=185.203.119.12 protocol=FTP domain=ftp.cloudsync-services.com action=STOR filename=vpn_config_backup.zip bytes_sent=86
```

5. I then wanted to count uploads to external destinations

`cat ftp_lab.log | cut -d' ' -f3 | sort | uniq -c`

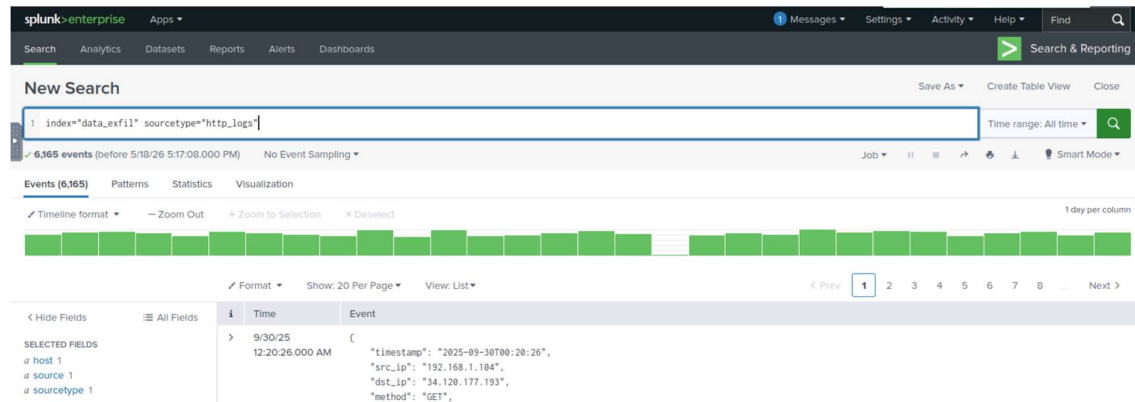
Here we can see repeated uploads where made to a single destination IP

```
File Edit View Search Terminal Help
ubuntu@tryhackme:~/Desktop/data_exfil/ftp_exfil$ cat ftp_lab.log | cut -d' ' -f3 | sort | uniq -c
400 dst=185.203.119.12
ubuntu@tryhackme:~/Desktop/data_exfil/ftp_exfil$
```

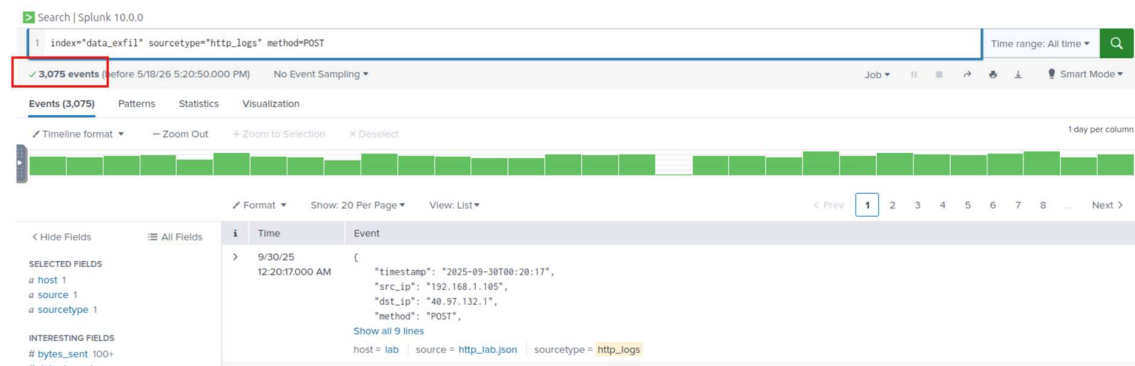
Data exfiltration via HTTP

Indicators of Attack

- Unusually large HTTP POST request to external or unexpected hosts
- HTTP requests to domains with low reputation
- Frequent small requests (Beaconing)
- Chunked or multipart transfers where multiple request compose a large file



The exfiltration attempts can be done via POST request so I will use `index="data_exfil" sourcetype="http_logs" method=POST`



Now lets find the average amounts of bytes transferred

Search | Splunk 10.0.0

```
1 index="data_exfil" sourcetype="http_logs" method=POST
2 | stats count avg(bytes_sent) max(bytes_sent) min(bytes_sent) by domain | sort - count
```

Time range: All time

3,075 events (before 5/18/26 5:22:48.000 PM) No Event Sampling

Events Patterns **Statistics (1)** Visualization

Show: 20 Per Page Format Preview: On

domain	count	avg(bytes_sent)	max(bytes_sent)	min(bytes_sent)
api.tools.io	329	375.22492401215886	450	300
dropbox.com	326	372.9141184294479	450	300
cdn.com	321	375.7819314641745	450	300
zoom.us	318	369.3993710691824	449	300
cloudflare.com	317	371.3501577287066	449	300
github.com	317	378.29968454258676	450	300
metrics.dev	304	372.91776315789474	450	301
datafeed.org	289	376.7681660899654	450	301
microsoft.com	289	378.4463667820069	449	300
update.net	264	374.2651515151515	450	300

Search | Splunk 10.0.0

```
1 index="data_exfil" sourcetype="http_logs" method=POST bytes_sent > 600 | table _time src_ip uri domain dst_ip bytes_sent | sort - bytes_sent
```

Time range: All time

1 event (before 5/18/26 5:24:21.000 PM) No Event Sampling

Events Patterns **Statistics (1)** Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	uri	domain	dst_ip	bytes_sent
2025-09-18 00:00:11	192.168.1.103	/v1/sync/upload	api.cloudsync-services.com	185.203.119.12	654

I will now correlate this information with the PCAP file to inspect further

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Frame	Info
1	9.45.000000	192.168.1.105	23.62.239.1	HTTP	451	✓	POST /update HTTP/1.1
2	3.000000	192.168.1.104	20.112.52.29	HTTP	454	✓	POST /update HTTP/1.1
3	6.000000	192.168.1.103	40.97.132.1	HTTP	90	✓	GET /v1/sync/upload HTTP/1.1
4	12.000000	192.168.1.105	23.62.239.1	HTTP	432	✓	POST /v1/sync/upload HTTP/1.1
5	22.000000	192.168.1.101	170.114.10.1	HTTP	83	✓	GET /status HTTP/1.1
6	27.000000	192.168.1.102	20.112.52.29	HTTP	87	✓	GET /v1/sync/upload HTTP/1.1
7	31.000000	192.168.1.102	23.45.67.89	HTTP	442	✓	POST /status HTTP/1.1
8	40.000000	192.168.1.103	23.45.67.89	HTTP	460	✓	POST /status HTTP/1.1
9	45.000000	192.168.1.104	34.120.177.193	HTTP	515	✓	POST /index.html HTTP/1.1
10	55.000000	192.168.1.103	162.125.66.1	HTTP	87	✓	GET /index.html HTTP/1.1
11	61.000000	192.168.1.104	23.62.239.1	HTTP	79	✓	GET /update HTTP/1.1
12	69.000000	192.168.1.104	34.120.177.193	HTTP	496	✓	POST /v1/sync/upload HTTP/1.1
13	78.000000	192.168.1.104	34.120.177.193	HTTP	88	✓	GET /index.html HTTP/1.1
14	86.000000	192.168.1.105	23.62.239.1	HTTP	82	✓	GET /status HTTP/1.1
15	87.000000	192.168.1.105	23.62.239.1	HTTP	70	✓	GET /update HTTP/1.1

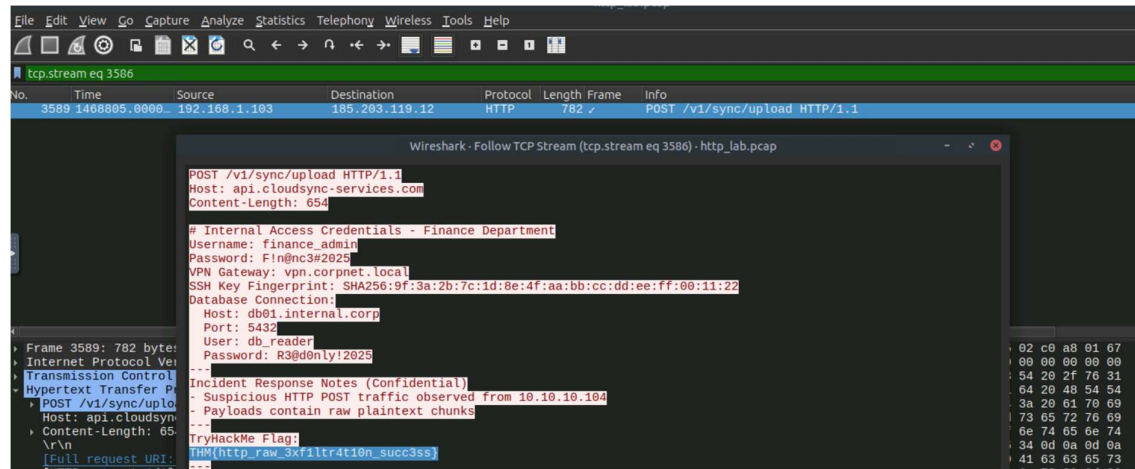
http.request.method == POST and frame.len > 500 to find traffic with a frame length greater than 500

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

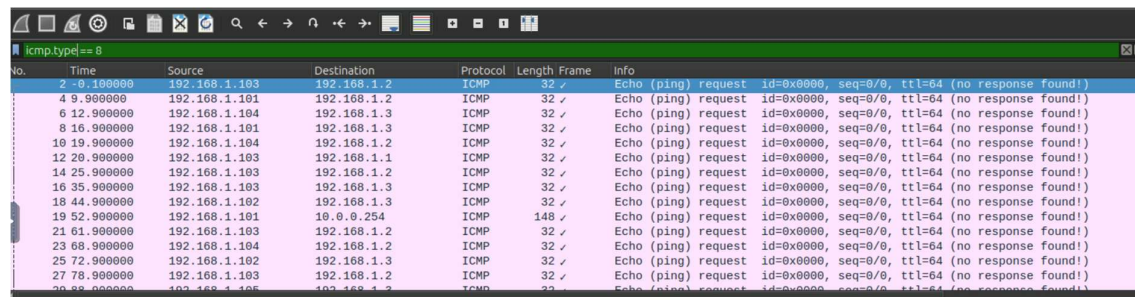
http.request.method == POST and frame.len > 500

No.	Time	Source	Destination	Protocol	Length	Frame	Info
9	45.000000	192.168.1.104	34.120.177.193	HTTP	515	✓	POST /index.html HTTP/1.1
21	125.000000	192.168.1.102	40.97.132.1	HTTP	512	✓	POST /update HTTP/1.1
22	126.000000	192.168.1.104	162.125.66.1	HTTP	554	✓	POST /update HTTP/1.1
28	154.000000	192.168.1.104	23.62.239.1	HTTP	547	✓	POST /status HTTP/1.1
32	183.000000	192.168.1.102	40.97.132.1	HTTP	550	✓	POST /index.html HTTP/1.1
42	254.000000	192.168.1.103	104.16.123.96	HTTP	514	✓	POST /v1/sync/upload HTTP/1.1
45	264.000000	192.168.1.101	44.236.72.1	HTTP	511	✓	POST /status HTTP/1.1
58	332.000000	192.168.1.103	23.45.67.89	HTTP	558	✓	POST /index.html HTTP/1.1
59	333.000000	192.168.1.103	104.16.123.96	HTTP	510	✓	POST /update HTTP/1.1
70	384.000000	192.168.1.102	20.112.52.29	HTTP	555	✓	POST /update HTTP/1.1
72	396.000000	192.168.1.104	23.45.67.89	HTTP	507	✓	POST /index.html HTTP/1.1
75	410.000000	192.168.1.105	23.45.67.89	HTTP	551	✓	POST /status HTTP/1.1
78	421.000000	192.168.1.104	34.120.177.193	HTTP	525	✓	POST /update HTTP/1.1
80	432.000000	192.168.1.101	104.16.123.96	HTTP	541	✓	POST /update HTTP/1.1
81	438.000000	192.168.1.103	40.97.132.1	HTTP	638	✓	POST /status HTTP/1.1

Still a lot of noise if we increase it we can filter it further 750



Data Exfiltration via ICMP



Now lets find packets with bigger data icmp.type == 8 and frame.len > 100

